

JAVA SPEZIAL

Immutable-Objekte in Java

Unveränderliche Objekte sicher einsetzen

Thread-Sicherheit, Fehlervermeidung, Praxisbeispiele
Konstruktion, Defensive Copies, with-Methoden

In diesem Skript:

- Was bedeutet **immutable** in Java?
- Regeln für das Design immutable Klassen
- Vergleich: mutable vs. immutable Beispielklasse
- Umgang mit veränderlichen Feldern (z.B. `Date`, Listen)
- Vorteile in nebenläufigen Anwendungen
- Best Practices aus Sicht eines Fachinformatikers

Inhaltsverzeichnis

1	Ziel der Lernunterlage	2
2	Was bedeutet immutable?	2
3	Warum immutable? (Praxis-Sicht eines Fachinformatikers)	2
4	Regeln für Immutable-Klassen in Java	2
5	Beispiel: Mutable vs. Immutable Person	3
5.1	Mutable Variante	3
5.2	Immutable Variante mit with-Methoden	3
6	Immutable bei veränderlichen Feldern	4
6.1	Beispiel mit java.util.Date	4
7	Immutable bei Listen und Arrays (Student-Beispiel)	5
7.1	Problem: scheinbar immutable mit veränderlicher Liste	5
7.2	Lösung: Immutable-Liste mit Defensive Copy	5
7.3	Alternative vor Java 10: Collections.unmodifiableList	6
7.4	Immutable mit Arrays: immer kopieren	7
8	Immutable und Threads	8
9	Best Practices aus der Praxis	8
10	Überblick: Wichtige immutable und mutable Typen in Java	8
10.1	Typische immutable Typen	9
10.2	Typische mutable Typen	9
11	Zusammenfassung	10
A	Schnellübersicht für die Prüfung	11
Anhang A:	Schnellübersicht für die Prüfung	11

1 Ziel der Lernunterlage

In dieser Unterlage halte ich die wichtigsten Punkte zum Thema **Immutable-Objekte in Java** fest. Der Fokus liegt auf praktischer Anwendbarkeit im Alltag eines Fachinformatikers Anwendungsentwicklung, insbesondere im Zusammenhang mit mehrschichtigen Business-Anwendungen und Nebenläufigkeit.

Merkkasten

Immutable bedeutet: **Zustand nach der Initialisierung ändert sich nicht mehr**. Jede scheinbare "Änderung" erzeugt ein **neues Objekt**.

2 Was bedeutet immutable?

Ein Objekt ist **immutable** (unveränderlich), wenn sein Zustand nach der Initialisierung nicht mehr geändert werden kann. Alle Felder behalten nach dem Anlegen der Instanz ihren Wert. Statt ein Objekt zu verändern, wird bei Änderungen ein **neues** Objekt erzeugt.

Typische Beispiele aus der Java-Standardbibliothek:

- **String**
- Wrapper-Klassen wie **Integer**, **Long**, **Double**
- Viele Klassen aus dem Paket **java.time** (z.B. **LocalDate**, **LocalDateTime**)

Stolperfallen

Auch wenn eine Klasse "harmlos" aussieht, ist sie nicht automatisch immutable. Wichtig sind: **final**-Felder, keine Setter und Schutz vor Änderungen über Referenzen auf interne Objekte.

3 Warum immutable? (Praxis-Sicht eines Fachinformatikers)

Aus Sicht eines Fachinformatikers sind immutable Klassen besonders nützlich, weil sie:

- **Thread-Sicherheit** vereinfachen: Wenn der Zustand nicht geändert werden kann, brauche ich oft keine Synchronisation (z.B. kein **synchronized**).
- **Fehler vermeiden**: Nebenwirkungen durch unerwartete Änderungen entfallen.
- **leichter zu verstehen** sind: Der Zustand einer Instanz ist nach dem Konstruktor klar definiert.
- **leicht zu testen** sind: Einmal erzeugt, kann sich ein Objekt im Test nicht heimlich verändern.

Nachteile sind z.B. zusätzlicher Speicher- und Objekt-Erzeugungsaufwand, wenn sehr viele neue Instanzen erzeugt werden. In vielen Geschäftsanwendungen ist das aber akzeptabel oder spielt praktisch keine Rolle.

4 Regeln für Immutable-Klassen in Java

Um eine Klasse sinnvoll immutable zu machen, halte ich mich an folgende Regeln:

1. Deklariere die Klasse nach Möglichkeit als **final**, damit sie nicht abgeleitet werden kann.
2. Setze alle Felder auf **private** und **final**.

3. Initialisiere alle Felder vollständig im Konstruktor.
4. Biete **keine Setter-Methoden** an.
5. Gib keine veränderlichen internen Objekte direkt zurück, sondern nur Kopien (**Defensive Copies**).
6. Nimm bei veränderlichen Parametern im Konstruktor ebenfalls Kopien, um den internen Zustand zu schützen.

Merkkasten

Konsequenz: Eine immutable Klasse ist wie ein Wert (Value Object). Wenn sich etwas ändert, erstelle ich einen neuen Wert statt den alten zu verändern.

5 Beispiel: Mutable vs. Immutable Person

5.1 Mutable Variante

```
1 public class PersonMutable {
2     private String name;
3     private int age;
4
5     public PersonMutable(String name, int age) {
6         this.name = name;
7         this.age = age;
8     }
9
10    public String getName() {
11        return name;
12    }
13
14    public void setName(String name) {
15        this.name = name;
16    }
17
18    public int getAge() {
19        return age;
20    }
21
22    public void setAge(int age) {
23        this.age = age;
24    }
25 }
```

Hier kann ich Name und Alter nachträglich ändern. In komplexen Anwendungen, vor allem nebenläufigen, kann das zu schwer nachvollziehbaren Fehlern führen.

5.2 Immutable Variante mit with-Methoden

```
1 public final class Person {
2     private final String name;
3     private final int age;
4
5     public Person(String name, int age) {
6         this.name = name;
7         this.age = age;
8     }
9 }
```

```

10 public String getName() {
11     return name;
12 }
13
14 public int getAge() {
15     return age;
16 }
17
18 // "Änderung" erzeugt neues Objekt
19 public Person withName(String newName) {
20     return new Person(newName, this.age);
21 }
22
23 public Person withAge(int newAge) {
24     return new Person(this.name, newAge);
25 }
26 }

```

Wichtige Punkte:

- Keine Setter – Änderungen erfolgen über Methoden wie `withAge()`, die ein **neues Objekt** zurückgeben.
- Felder sind `private final` und werden nur im Konstruktor gesetzt.
- Die Klasse ist `final`, um Vererbung zu verhindern.

6 Immutable bei veränderlichen Feldern

Schwieriger wird es bei Feldern, die selbst veränderlich sind, zum Beispiel `java.util.Date` oder `Listen`.

6.1 Beispiel mit `java.util.Date`

```

1 import java.util.Date;
2
3 public final class Event {
4     private final String title;
5     private final Date date;
6
7     public Event(String title, Date date) {
8         this.title = title;
9         // Defensive Copy im Konstruktor
10        this.date = new Date(date.getTime());
11    }
12
13    public String getTitle() {
14        return title;
15    }
16
17    public Date getDate() {
18        // Defensive Copy im Getter
19        return new Date(date.getTime());
20    }
21 }

```

Ohne die Kopien könnte von außen über das zurückgegebene `Date`-Objekt der interne Zustand geändert werden.

Stolperfallen

Wenn ein Getter eine interne, veränderliche Collection oder ein `Date` direkt zurückgibt, ist die Klasse **nicht** mehr wirklich immutable – auch wenn alle Felder `final` sind.

7 Immutable bei Listen und Arrays (Student-Beispiel)

In echten Fachinformatiker-Projekten enthalten Domänenklassen oft Sammlungen, zum Beispiel eine Liste von Fähigkeiten (Skills) eines Students. Genau hier passieren viele Fehler: Die Klasse wirkt immutable, ist es aber nicht, weil eine Liste oder ein Array "durchgereicht" wird.

7.1 Problem: scheinbar immutable mit veränderlicher Liste

Ausgangspunkt ist eine Klasse mit einer `List<String>`:

```
1 public final class StudentImmutable {
2     private final String name;
3     private final String vorname;
4     private final LocalDate geburtsdatum;
5
6     private final List<String> skills; // veränderliche Collection
7
8     public StudentImmutable(String name, String vorname,
9         LocalDate geburtsdatum,
10        List<String> skills) {
11         this.name = name;
12         this.vorname = vorname;
13         this.geburtsdatum = geburtsdatum;
14         this.skills = skills; // Problem!
15     }
16
17     public String getName() { return name; }
18     public String getVorname() { return vorname; }
19     public LocalDate getGeburtsdatum() { return geburtsdatum; }
20
21     public List<String> getSkills() {
22         return skills; // Problem!
23     }
24 }
```

Auf den ersten Blick wirkt alles immutable: `final`-Felder, keine Setter, `final`-Klasse. Trotzdem ist die Klasse nicht wirklich unveränderlich:

- Die übergebene `skills`-Liste wird direkt gespeichert. Wer die Ursprungsliste noch hat, kann sie nachträglich ändern.
- Über `getSkills().add(NNeuer Skill)` kann der Aufrufer direkt in die interne Liste schreiben.

Stolperfallen

Eine Klasse ist **nicht** automatisch immutable, nur weil alle Felder `final` sind. Bei veränderlichen Objekten (Listen, Arrays, `Date`,...) muss ich mit **Defensive Copies** arbeiten – sowohl im Konstruktor als auch in den Gettern.

7.2 Lösung: Immutable-Liste mit Defensive Copy

Ab Java 10 kann ich mit `List.copyOf(...)` eine eigene, wirklich unveränderliche Kopie der Liste erzeugen:

```

1 import java.time.LocalDate;
2 import java.util.List;
3
4 public final class StudentImmutable {
5     private final String name;
6     private final String vorname;
7     private final LocalDate geburtsdatum;
8     private final List<String> skills;
9
10    public StudentImmutable(String name, String vorname,
11        LocalDate geburtsdatum,
12        List<String> skills) {
13        this.name = name;
14        this.vorname = vorname;
15        this.geburtsdatum = geburtsdatum;
16        // Defensive Copy + Einfrieren
17        this.skills = List.copyOf(skills);
18    }
19
20    public String getName() { return name; }
21    public String getVorname() { return vorname; }
22    public LocalDate getGeburtsdatum() { return geburtsdatum; }
23
24    public List<String> getSkills() {
25        // bereits immutable
26        return skills;
27    }
28
29    // Beispiel für with-Methode
30    public StudentImmutable withName(String name) {
31        return new StudentImmutable(name, this.vorname,
32            this.geburtsdatum, this.skills);
33    }
34 }

```

Wichtige Effekte:

- Änderungen an der ursprünglichen Liste (die dem Konstruktor übergeben wurde) haben keine Auswirkung mehr auf den Student.
- `getSkills()` liefert eine unveränderliche Liste: `getSkills().add(...)` wirft eine `UnsupportedOperationException`.

Merkkasten

Bei Collections reicht `final` nicht aus. Für echte Immutable-Objekte brauche ich:

- Im Konstruktor: **Kopie** der Liste anlegen und unveränderlich machen (z.B. `List.copyOf(...)`).
- Im Getter: niemals eine modifizierbare interne Liste direkt zurückgeben.

7.3 Alternative vor Java 10: `Collections.unmodifiableList`

Wenn `List.copyOf` noch nicht verfügbar ist (z.B. Java 8), arbeite ich mit `Collections.unmodifiableList` und einer zusätzlichen Kopie:

```

1 import java.time.LocalDate;
2 import java.util.ArrayList;
3 import java.util.Collections;
4 import java.util.List;

```

```

5
6 public final class StudentImmutableLegacy {
7     private final String name;
8     private final String vorname;
9     private final LocalDate geburtsdatum;
10    private final List<String> skills;
11
12    public StudentImmutableLegacy(String name, String vorname,
13        LocalDate geburtsdatum,
14        List<String> skills) {
15        this.name = name;
16        this.vorname = vorname;
17        this.geburtsdatum = geburtsdatum;
18        // eigene Kopie + unmodifiable-View
19        this.skills = Collections.unmodifiableList(
20            new ArrayList<>(skills)
21        );
22    }
23
24    public List<String> getSkills() {
25        return skills; // unmodifiable View auf die Kopie
26    }
27 }

```

Stolperfallen

`Collections.unmodifiableList(list)` allein schützt nur vor direkter Modifikation über diese View.

Wenn ich die **gleiche** Liste noch irgendwo anders verändere, ändert sich auch der Inhalt der unmodifiable-View. Darum ist die Kombination aus `new ArrayList<>(...)` und `Collections.unmodifiableList(...)` wichtig.

7.4 Immutable mit Arrays: immer kopieren

Bei Arrays gibt es keinen eigenen „unmodifiable“-Wrapper. Hier erreiche ich Immutability, indem ich konsequent Kopien verwende – sowohl im Konstruktor als auch im Getter.

```

1 import java.time.LocalDate;
2
3 public final class StudentImmutableArray {
4     private final String name;
5     private final String vorname;
6     private final LocalDate geburtsdatum;
7     private final String[] skills;
8
9     public StudentImmutableArray(String name, String vorname,
10        LocalDate geburtsdatum,
11        String[] skills) {
12        this.name = name;
13        this.vorname = vorname;
14        this.geburtsdatum = geburtsdatum;
15        // defensive Copy im Konstruktor
16        this.skills = skills == null ? null : skills.clone();
17    }
18
19    public String getName() { return name; }
20    public String getVorname() { return vorname; }
21    public LocalDate getGeburtsdatum() { return geburtsdatum; }
22 }

```



```

23 public String[] getSkills() {
24     // defensive Copy im Getter
25     return skills == null ? null : skills.clone();
26 }
27 }

```

Damit gilt:

- Änderungen am ursprünglichen Array (außerhalb der Klasse) haben keinen Einfluss auf den Student.
- Änderungen am von `getSkills()` zurückgegebenen Array betreffen nur die Kopie, nicht den internen Zustand.

Merkkasten

Bei Arrays gibt es keine eingebaute Immutable-Unterstützung.
Regel:

- Im Konstruktor: übergebenes Array immer klonen.
- Im Getter: internes Array nie direkt zurückgeben, sondern ebenfalls klonen.

8 Immutable und Threads

In nebenläufigen Anwendungen (z.B. Webserver, Hintergrund-Worker) vereinfacht Immutable-Design die Synchronisation:

- Immutable-Objekte können von beliebig vielen Threads gleichzeitig gelesen werden, ohne dass ich sperren muss.
- Risiko von **Race Conditions** sinkt deutlich.
- Besonders bei Konfigurationsobjekten, DTOs oder Value-Objekten lohnt sich dieser Ansatz.

In der Praxis kombiniere ich immutable Objekte gern mit **Buildern** oder Fabrikmethoden, um die Erzeugung komfortabel zu halten.

9 Best Practices aus der Praxis

- Immutable Klassen gezielt für Domänenwerte einsetzen (z.B. Geldbeträge, E-Mail-Adressen, IDs).
- Für Konfigurationsdaten und Ergebnisse von Berechnungen bevorzugt immutable Strukturen nutzen.
- Bei Performanceproblemen prüfen, ob wirklich die Objekt-Erzeugung das Problem ist – oft ist das nicht der Fall.
- Konsequenz auf Settermethoden verzichten, wenn das Objekt als Wertobjekt gedacht ist.

10 Überblick: Wichtige immutable und mutable Typen in Java

Zum Abschluss fasse ich zentrale Java-Typen zusammen, die im Zusammenhang mit Immutable-Design für Fachinformatiker wichtig sind. Dabei unterscheide ich zwischen typischerweise *immutable* und *mutable* Typen.

10.1 Typische immutable Typen

Diese Typen ändern ihren Zustand nach der Initialisierung nicht mehr. "Änderungen" erzeugen neue Objekte.

- **String**
 - Inhalt kann nicht geändert werden.
 - Operationen wie `concat()`, `substring()` etc. liefern immer **neue** String-Objekte.
- **Wrapper-Klassen:** `Integer`, `Long`, `Double`, `Boolean`,...
 - Repräsentieren einen Wert (z.B. eine Zahl) und sind immutable.
- **java.time-API:** z.B. `LocalDate`, `LocalDateTime`, `Instant`
 - Moderne Datums-/Zeit-Klassen sind bewusst immutable designet.
 - Methoden wie `plusDays()` geben ein **neues** Objekt zurück.
- **Eigene Value Objects**
 - Domänentypen wie Geldbeträge (`Money`), E-Mail-Adressen oder IDs können als immutable Klassen umgesetzt werden.
 - Sie verhalten sich wie Werte und passen gut zu den in dieser Unterlage vorgestellten Regeln.

10.2 Typische mutable Typen

Diese Typen können nach der Initialisierung ihren Zustand ändern. Hier muss ich besonders aufpassen, wenn ich immutable Klassen bauen möchte.

- **Collections aus `java.util`** (`List`, `Set`, `Map` und Implementierungen wie `ArrayList`, `HashSet`, `HashMap`,...)
 - Standard-Collections sind **immer mutable**.
 - Sie müssen in immutable Klassen durch **Defensive Copies** und `unmodifiable/immutable` Varianten geschützt werden (z.B. `List.copyOf(...)`).
- **Arrays** (z.B. `String[]`)
 - Länge und Elemente können geändert werden (`array[i] = ...`).
 - In immutable Klassen sind Kopien im Konstruktor und im Getter Pflicht (`clone()`).
- **StringBuilder und StringBuffer**
 - Speziell dafür gedacht, Strings effizient zu verändern (Anhängen, Einfügen, Löschen).
 - **StringBuilder:** nicht thread-sicher, aber schnell.
 - **StringBuffer:** synchronisiert, thread-sicherer, aber langsamer.
 - Im Gegensatz zu `String` sind diese Klassen bewusst **mutable**.
- **Alte Datums-API:** `java.util.Date`, `java.util.Calendar`
 - Zustand (Zeitpunkt, Felder) kann nachträglich verändert werden.
 - Für neue Entwicklungen sollte bevorzugt die immutable `java.time`-API verwendet werden.

Merkkasten

Faustregel:

- Wenn ein Typ Methoden hat, die seinen Zustand direkt ändern (z.B. `set...()`, `append()`, `add()`, `remove()`), ist er in der Regel **mutable**.
- Immutable-Typen geben bei "Änderungen" immer neue Objekte zurück und besitzen keine Setter.

Stolperfallen

Auch wenn ich in meiner eigenen Klasse alle Felder auf **final** setze, kann sie trotzdem **nicht** wirklich immutable sein, wenn sie mutable Typen wie Collections, Arrays oder **Date** ungeschützt nach außen durchreicht.

Erst durch konsequente **Defensive Copies** im Konstruktor und in den Gettern (wie im `StudentImmutable`-Beispiel) werden solche Klassen wirklich unveränderlich.

11 Zusammenfassung

Aus Sicht eines Fachinformatikers bieten Immutable-Objekte in Java vor allem Vorteile in Bezug auf:

- Verständlichkeit des Codes
- Fehlervermeidung
- Thread-Sicherheit

Wenn ich Wertobjekte und Konfigurationen immutable designe, wird mein Code robuster und einfacher zu warten.

A Schnellübersicht für die Prüfung

Immutable-Objekte – Kernidee

- **Immutable:** Zustand ändert sich nach der Initialisierung nicht mehr.
- Jede scheinbare "Änderung" erzeugt ein **neues Objekt** (z.B. mit `with...`-Methoden).
- Kein **setter**, keine Methoden, die Felder direkt verändern.

Regeln für immutable Klassen

- Klasse möglichst als **final** deklarieren.
- Alle Felder: **private** und **final**.
- Vollständige Initialisierung im Konstruktor.
- Keine Setter-Methoden anbieten.
- Bei veränderlichen Typen (Listen, Arrays, Date): **Defensive Copies** im Konstruktor und in Gettern.

Wichtige Typen – Überblick

Typisch immutable

- `String`
- Wrapper: `Integer`, `Long`, `Double`,...
- `java.time`: `LocalDate`, `LocalDateTime`,...
- Eigene Value Objects (z.B. `Money`, E-Mail-Adresse)

Typisch mutable

- Collections: `List`, `Set`, `Map`, `ArrayList`, `HashMap`,...
- Arrays: z.B. `String[]` (Elemente veränderbar)
- `StringBuilder`, `StringBuffer`
- Alte Datums-API: `java.util.Date`, `Calendar`

Immutable mit Collections und Arrays

Listen (ab Java 10)

```
1 // Defensive Copy + immutable
2 this.skills = List.copyOf(skills);
3
4 public List<String> getSkills() {
5     return skills; // unveränderlich
6 }
```

Listen (Java 8-kompatibel)

```
1 this.skills = Collections.unmodifiableList(  
2 new ArrayList<>(skills)  
3 );
```

Arrays

```
1 // im Konstruktor  
2 this.skills = skills == null ? null : skills.clone();  
3  
4 // im Getter  
5 public String[] getSkills() {  
6 return skills == null ? null : skills.clone();  
7 }
```

Typisches Muster: with-Methoden

```
1 public final class Person {  
2 private final String name;  
3 private final int age;  
4  
5 public Person(String name, int age) {  
6 this.name = name;  
7 this.age = age;  
8 }  
9  
10 public Person withName(String newName) {  
11 return new Person(newName, this.age);  
12 }  
13  
14 public Person withAge(int newAge) {  
15 return new Person(this.name, newAge);  
16 }  
17 }
```

Stolperfallen – ganz kurz

- **Final reicht nicht:** `private final List<>` ist nicht immutable, wenn die Liste direkt durchgereicht wird.
- **`Collections.unmodifiableList`** ohne eigene Kopie: Änderungen an der Ursprungs-Liste ändern auch die View.
- Getter, die `Date`, `Collections` oder `Arrays` direkt zurückgeben, machen die Klasse von außen veränderbar.
- Bei Immutable-Objekten sind Exceptions wie `UnsupportedOperationException` bei Änderungsversuchen **erwünscht** – sie schützen den Zustand.